De Figma a Android · Pantalla «Carrito»

App «Pequeños Pasos» (tienda de ropa infantil) · PMDM

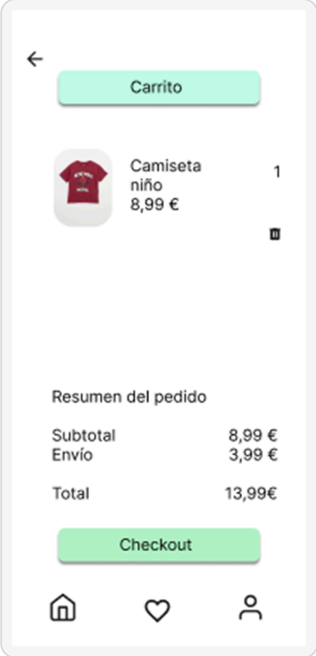
Alumno	Daniel Radulescu
Prototipo (Figma)	https://www.figma.com/proto/umNYz7ZW0U2snS897ic9yl/T1_Daniel--Copia-?node-id=0-1
Archivo (Dev Mode)	https://www.figma.com/design/umNYz7ZW0U2snS897ic9yl/T1_Daniel--Copia-?node-id=0-1&m=dev
Pantalla elegida	4 · Carrito. Tiene iconos (flecha atrás, papelera, barra inferior), una fotografía (camiseta), tres estilos de texto (título, destacado y cuerpo) y un botón (Checkout).
Proyecto	Java · Empty Views Activity · API mínima 28 · ViewBinding · actividad `CarritoActivity`

A1 · Análisis del diseño

`[NP]` no se programa como vista (pasa a ser un atributo o un drawable) `[SUP]` superpuesto a otro elemento

`[REP]` se repite

```
4 - Carrito (frame)                  → ConstraintLayout
├── Barra superior
│   ├── Flecha atrás                 → ImageButton
│   └── Cabecera
│       ├── Rectángulo menta [NP][SUP]  → fondo (drawable)
│       │   └── Sombra [NP]          → elevation
│       └── «Carrito» [SUP]          → TextView
├── Producto del carrito [REP]       → fila de la lista
│   ├── Foto de la camiseta          → ImageView
│   │   └── Máscara redondeada [NP]  → forma (shape)
│   ├── «Camiseta niño»              → TextView
│   ├── «8,99 €»                     → TextView
│   ├── «1» (cantidad)               → TextView
│   └── Papelera                     → ImageButton
├── Resumen del pedido
│   ├── «Resumen del pedido»         → TextView
│   ├── Fila Subtotal [REP]          → etiqueta + importe
│   │   ├── «Subtotal»               → TextView
│   │   └── «8,99 €»                 → TextView
│   ├── Fila Envío [REP]             → etiqueta + importe
│   └── Fila Total [REP]             → etiqueta + importe
├── Botón Checkout                   → Button (una vista)
│   ├── Rectángulo verde [NP][SUP]   → backgroundTint + radio
│   │   └── Sombra [NP]              → elevation
│   └── «Checkout» [SUP]             → texto del botón
└── Barra inferior                   → BottomNavigationView
    ├── Casa (Inicio) [REP]          → elemento del menú
    ├── Corazón (Favoritos) [REP]    → elemento del menú
    └── Persona (Perfil) [REP]       → elemento del menú
```



Pantalla 4 · Carrito en Figma

Comparación con el panel de capas

- **Cabecera y botón:** en Figma son dos capas (rectángulo + texto) y en Android una sola vista. El rectángulo pasa a ser el fondo, y la sombra, que en Figma es un efecto, pasa a ser `elevation`.
- **Iconos:** cada uno es un grupo de vectores en Figma y se convierte en un único VectorDrawable.
- **Barra inferior:** en Figma son tres capas sueltas; en Android, un `BottomNavigationView` con un menú de tres elementos.
- **Repetición:** la fila del producto se repetiría por cada artículo (en el futuro, elemento de una lista) y las tres filas del resumen comparten estructura.

A3 · Ficha de especificaciones

Frame de 393 × 852 px. A 1x, 1 px de Figma equivale a 1 dp en Android, y en los textos a 1 sp.

Colores

Nombre funcional	Uso	Figma	Opacidad	colors.xml
`color_primario`	Fondo del botón Checkout	#AFF4C6	100 %	#AFF4C6
`color_cabecera`	Fondo de la cabecera	#BEFBE6	100 %	#BEFBE6
`fondo_pantalla`	Fondo del frame y de la barra inferior	#FFFFFF	100 %	#FFFFFF
`texto_principal`	Todos los textos	#000000	100 %	#000000
`icono`	Flecha atrás y papelera	#1D1B20	100 %	#1D1B20
`icono_navegacion`	Iconos de la barra inferior	#1E1E1E	100 %	#1E1E1E
`sombra`	Sombra de la cabecera y del botón	#000000	25 %	**#40000000**

Conversión de la transparencia: Android usa el formato #AARRGGBB. 25 % → 0,25 × 255 = 63,75 ≈ 64 = 0x40, así que el color queda #40000000.

Tipografía

En el diseño original todos los textos tenían el mismo estilo: Inter Regular, 20 px, interlineado Auto y espaciado 0 %. Para tener jerarquía y cumplir los tres estilos definí estos:

Estilo (styles.xml)	Dónde se usa	Fuente	Tamaño	Interlineado	Espaciado
`Texto.Titulo`	«Carrito»	Inter SemiBold (600)	24 px → **24 sp**	Auto 121 % → 24 × 1,21 = 29,04 → **29 sp**	0 % → 0 em
`Texto.Destacado`	«Resumen del pedido», «Total», importe total, «Checkout»	Inter Medium (500)	20 px → **20 sp**	20 × 1,21 = 24,2 → **24 sp**	0 % → 0 em
`Texto.Cuerpo`	Nombre y precio del producto, cantidad, «Subtotal», «Envío» e importes	Inter Regular (400)	18 px → **18 sp**	18 × 1,21 = 21,78 → **22 sp**	0 % → 0 em

Los textos van en sp para que respeten el tamaño de letra que elija el usuario. El interlineado Auto de Inter es el 121 % del tamaño; se pasa a sp y se aplica con `android:lineHeight`, disponible desde la API 28. El espaciado entre letras de Figma (%) se divide entre 100 para obtener em (`android:letterSpacing`).

Medidas

Elemento	Figma (px)	Android
Cabecera	266 × 44, x 64, radio 5	márgenes laterales 64 dp, alto 44 dp, radio 5 dp
Botón Checkout	266 × 44, x 64, radio 5	márgenes laterales 64 dp, alto 44 dp, radio 5 dp
Sombra (cabecera y botón)	X 0, Y 4, desenfoque 4, #000000 25 %	`elevation` 4 dp + color de sombra `@color/sombra` (API 28)
Foto del producto	78 × 106, x 56, radio 16	78 × 106 dp, margen inicial 56 dp, esquinas 16 dp
Separación foto–texto	24	24 dp
Resumen del pedido	margen izquierdo 54, derecho 41	54 dp / 41 dp
Flecha atrás	30 × 30	icono de 30 dp dentro de un área táctil de 48 dp
Papelera	24 × 24	icono de 24 dp dentro de un área táctil de 48 dp
Iconos de la barra inferior	40 × 40	`itemIconSize` 40 dp

Las flechas y la papelera miden menos de 48 dp, el mínimo táctil de Android, así que se les añade relleno (padding) para ampliar la zona pulsable. En el diseño el total era 13,99 €, pero 8,99 + 3,99 = 12,98 €; en strings.xml está corregido.

A6 · Inventario de assets

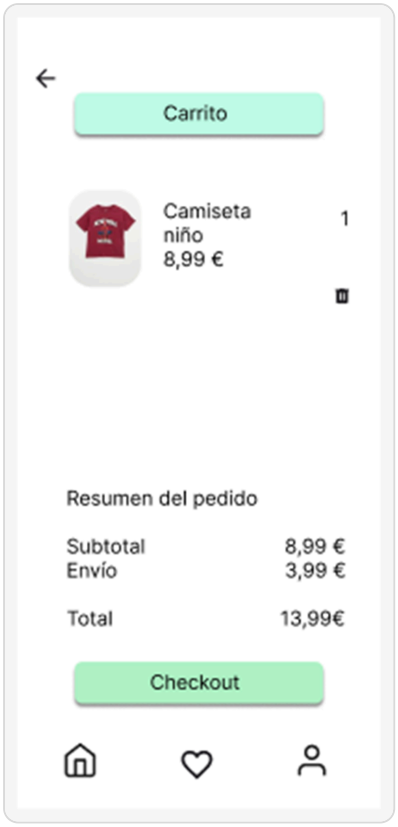
Recurso	Qué es	Formato	Carpeta	Tamaño
`ic_volver`	Flecha atrás	VectorDrawable	drawable	391 B
`ic_eliminar`	Papelera	VectorDrawable	drawable	403 B
`ic_inicio`	Casa (barra inferior)	VectorDrawable	drawable	703 B
`ic_favoritos`	Corazón (barra inferior)	VectorDrawable	drawable	583 B
`ic_perfil`	Persona (barra inferior)	VectorDrawable	drawable	722 B
`bg_cabecera`	Rectángulo menta de la cabecera	Shape (XML)	drawable	300 B
`img_camiseta_nino`	Foto de la camiseta	WebP con pérdida (calidad 75 %)	drawable-xxhdpi	234 × 318 px · 5.444 B
`inter_regular`, `inter_medium`, `inter_semibold`	Fuente Inter en tres pesos (Google Fonts, licencia OFL)	TTF + familia `inter.xml`	font	≈ 325 KB cada una

Decisiones

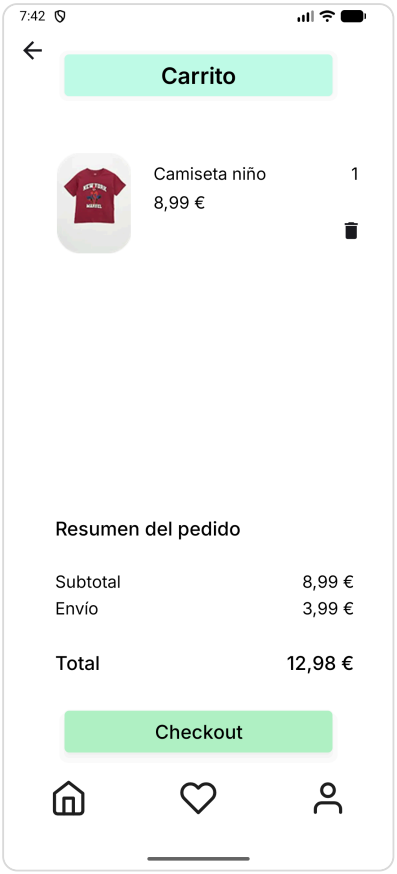
- **Iconos como vectores.** Un VectorDrawable se ve nítido en cualquier densidad con un solo archivo de menos de 1 KB, y su color sale de `colors.xml`.
- **Foto en WebP y en una sola densidad.** La foto se ve a 78 × 106 dp. En xxhdpi (×3) son 234 × 318 px, y Android la reduce para las densidades menores. Pesaba 88.295 B en PNG y 5.444 B en WebP, un 94 % menos.
- **Fondo de la cabecera como drawable.** El rectángulo no es una imagen, sino una forma XML con el color y el radio de la ficha.

Captura del emulador

La app compila y se ejecuta en el emulador Medium Phone (1080 × 2400, API 37). Para comprobar los recursos, la pantalla usa los colores, textos, estilos, fuentes, iconos y la foto de las actividades A5 y A6.



Diseño en Figma



App en ejecución

A7 · Reflexión

¿Qué decisión del análisis o de la preparación en Figma te ha resultado más difícil? ¿Por qué?
Lo más difícil fue decidir qué capas se programan y cuáles no. En Figma la cabecera y el botón son un rectángulo con un texto encima, y al principio pensaba hacer dos vistas. Al analizarlo vi que en Android es una sola: el rectángulo pasa a ser el fondo y la sombra un atributo de elevación. También me costó la tipografía. Al revisar los textos descubrí que todos tenían el mismo estilo, así que tuve que crear una jerarquía de tres (título, destacado y cuerpo) para que la pantalla se lea mejor.

Si mañana el cliente modifica el color principal, ¿qué tendrías que cambiar? ¿Y si no hubieras usado recursos?
Con recursos solo cambio una línea, el valor de `color_primario` en `colors.xml`, y se actualizan a la vez el botón, el tema y cualquier vista que lo use. Sin recursos tendría que buscar el código #AFF4C6 en todos los layouts y estilos y cambiarlo uno a uno, con el riesgo de olvidar alguno y que la app quede con colores distintos.

¿Cuánto ocupaba tu imagen en PNG y cuánto en WebP? ¿Qué conclusión sacas?
En PNG ocupaba 88.295 bytes (86,2 KB) y en WebP 5.444 bytes (5,3 KB): unas 16 veces menos (un 94 % menos), y a ese tamaño no se nota la diferencia. Mi conclusión es que las fotos deben ir en WebP y solo en la densidad necesaria, porque así el APK y la descarga son mucho más ligeros sin perder calidad visible.